

NCCT Attendance Kiosk — Hardware Architecture Review

Self-contained physical attendance unit · 2× ESP32 · NFC + camera + physical feedback · Reviewed against the actual repository, 15 Sep 2026

VERDICT: FEASIBLE WITH CHANGES

The wireless, no-laptop design is achievable — but one blocking capability does not exist in the repo today and must be resolved before any firmware is written.

● BLOCKER — nothing in this system can turn a JPEG into a face match.

Your flow says “ESP32-CAM sends the image to the existing Express backend.” No such endpoint exists, and neither does the capability behind it.

POST /timetable/:sessionId/kiosk-face-checkin accepts { trainee_id, embedding: number[1024] } — an **embedding, not an image**. The backend never sees a picture; it only recomputes cosine similarity.

The only component that can produce that descriptor is @vladmandic/human running **in a browser with WebGL**. Verified: it appears in apps/web/package.json only, and is absent from apps/api .

DECISIONS.md #16 records that server-side extraction was already attempted and failed: @tensorflow/tfjs-node would not install (no prebuilt binary for that Node/ABI; native compile needs VS C++ build tools), and the WASM Node build returned **zero face detections** on a clear multi-face photo, reproducibly, even at minConfidence: 0.05 . The ESP32-CAM cannot substitute — ESP-WHO uses a different, incompatible embedding space.

The decision to make first

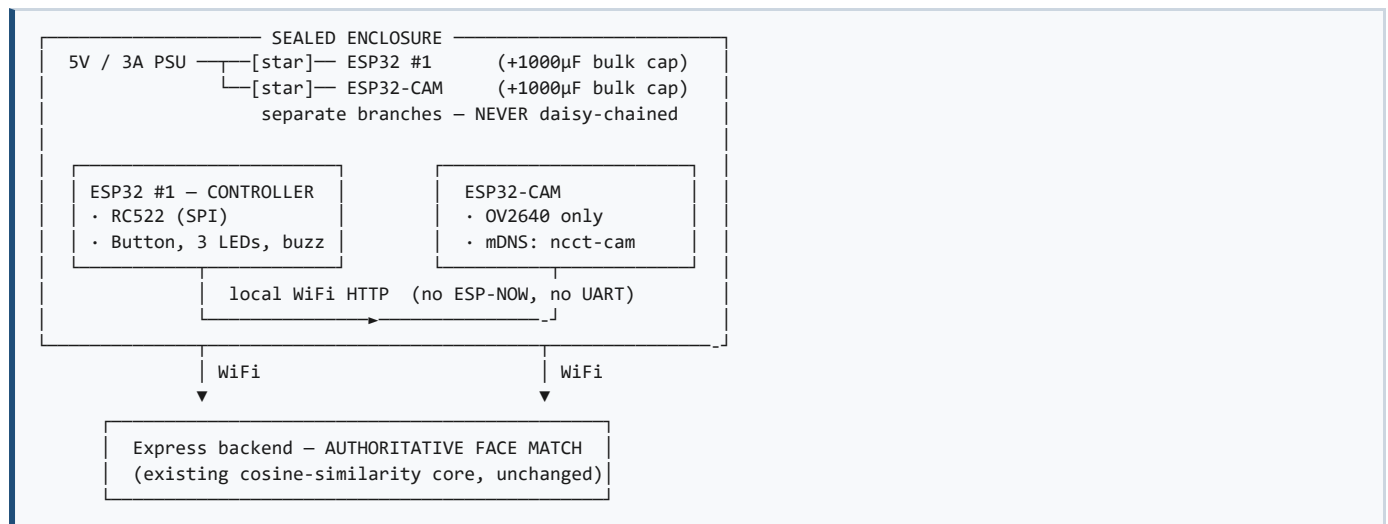
Path	What it is	Keeps enrollments?	Effort	Risk
B recommended first test	Retry tfjs-node on Linux. The #16 failure was on Windows. Render runs Linux x64, where prebuilt tfjs-node binaries generally exist. Same model, same 1024-d.	✓ Yes — nothing else changes	~30 min to test	Cheap to try; may still fail. Render free tier = 512 MB RAM, tight.
C safest fallback	Put the browser inside the enclosure. Pi Zero 2 W / Pi 4 / spare Android in kiosk mode, sealed in the box. With the new wireless design it needs no USB — it reaches both ESP32s over WiFi.	✓ Yes — zero biometric change	~½ day	Lowest risk by far. Cost ≈ ₹0–5,000.
A	onnxruntime-node + InsightFace buffalo_L (DECISIONS #5’s documented fallback). Prebuilt binaries exist for Win + Linux.	✗ No — 512-d vs current 1024-d	1–2 days	Vector-column migration, everyone re-enrolls , ~170 MB model vs 512 MB tier = real OOM risk.

Recommendation: spend 30 minutes testing Path B. If it works you get the pure-firmware architecture with nothing sacrificed. If it fails, take Path C and ship.

Trap in Path A: enrollment and verification *must* use the same model. Server-side buffalo_L verification forces enrollment server-side too — that doubles the scope.

Path C deserves a fair hearing. Your real requirement is “no teacher laptop, no per-student interaction, one sealed unit.” A Pi inside the box satisfies all three. It compromises only the wording “no computer.”

Recommended architecture



Under **Path C**, add the Pi/phone inside the enclosure; the ESP32 exposes `GET /events` which the browser polls every 500 ms. Everything else is identical.

ESP-NOW vs local HTTP — recommendation: local HTTP, not ESP-NOW

- **ESP-NOW's one real advantage is void here.** It shines when devices talk *without* infrastructure — but both your boards already require the AP (CAM for uploads, ESP32 for UID lookup). If the AP is down the system is dead either way.
- **It adds a genuine failure mode.** ESP-NOW and WiFi STA coexist only on the *same channel*, dictated by the AP. You cannot force the channel without breaking the STA link. If the AP ever changes channel (band steering, DFS, router reboot), **ESP-NOW silently stops delivering while WiFi keeps working** — exactly the intermittent failure class that has already cost you days on this project.
- **Local HTTP is strictly simpler:** the ESP32-CAM *already runs an HTTP server*. Triggering it is one `HTTPClient` call — no second radio protocol, no MAC registration, and testable with `curl` during development.
- **Its one real weakness — DHCP address churn** (which has bitten you twice: 10.20.111.50 → 192.168.1.8) — is fixed by one line: `MDNS.begin("ncct-cam")`.
- **Option B (backend-relayed) is worse than both:** it routes a local trigger through the internet and back for a decision made 10 cm away.

How the ESP32 learns PASS/FAIL — chain the synchronous calls

```
ESP32 --GET /capture-and-verify?trainee=X&session=Y--> CAM [held open]
CAM --POST image-----> Backend [held open]
CAM <--PASS/FAIL----- Backend
ESP32 <----- PASS | FAIL | DUP | RETRY ----- CAM
```

The ESP32's own HTTP response body is the verdict. No result endpoint, no polling, no new protocol. Set `HTTPClient::setTimeout()` ≈ 12 s. **MQTT and WebSockets are unnecessary** — you have exactly one request/response interaction, which is what HTTP already is. If chaining proves flaky in testing, *then* fall back to 500 ms polling — not before.

Hardware — two must-fix pin conflicts

① **GPIO 5 for RC522 SS → change back to GPIO 21.** The repo’s bench-tested sketch uses `SS_PIN 21`. GPIO 5 is a **strapping pin** (must be HIGH at boot; emits PWM at boot on many boards). SS idles HIGH so it usually survives — but this trades a hardware-verified wiring for an unverified one with a boot-strap dependency, for no gain.

② **If you use PN532 instead of RC522, your LED map is broken.** The PN532 sketch uses SCK=14, MISO=12, MOSI=13, **SS=26 — which collides with your Red LED**. Worse, **GPIO 12 is the MTDI strapping pin** (must be LOW at boot; a pulled-up MISO there can stop the board booting at all). **Pick RC522 and drop PN532 from this build**, or re-map both.

Function	GPIO	Notes
RC522 SS	21	⚠ changed from your proposed GPIO 5 — matches tested sketch
RC522 SCK / MOSI / MISO	18 / 23 / 19	VSPI defaults — unchanged
RC522 RST	22	unchanged
Green / Red / Yellow LED	25 / 26 / 27	✅ safe · 220 Ω each (~6 mA, well under 20 mA/pin limit)
Buzzer	32	✅ safe · via NPN transistor (see below)
Capture button	33	✅ safe — full GPIO with internal pull-up. (Input-only pins are 34/35/36/39 — avoid for buttons, no pull-ups.)
ESP32-CAM pins	—	✅ completely untouched by this design

- **Buzzer:** active buzzer via NPN (BC547/2N2222) + 1 kΩ base resistor. Direct-driving 25–30 mA from a GPIO exceeds the recommended limit. Add a flyback diode if magnetic (not piezo).
- **Button:** `INPUT_PULLUP`, switch to GND, **50 ms software debounce** — without it one press sends several CAPTUREs.

Power — and a conceptual correction

“No shared power” conflates two different things. Your brownout was caused by powering the CAM **from the other board’s regulator**. Feeding both from **one upstream 5 V supply via separate star-wired branches** is a completely different arrangement, and is safe and standard.

- ESP32 DevKit ≈ 80–260 mA (≈500 mA TX peaks) · ESP32-CAM ≈ 180–310 mA, **peaking 600–800 mA** on WiFi TX + capture
- **One 5 V / 3 A supply**, star-wired (both branches back to the input terminal, never in series), **1000 µF bulk cap at each board’s 5 V pin**
- **Common ground:** not *required* (no signal wires between boards), but one PSU means shared ground anyway — harmless at these currents. Don’t engineer around it.

Software changes

Unchanged: cosine-similarity matching, threshold/fallback semantics, consent enforcement, `face_embeddings / attendance_records` schema, camera init config, RC522 SPI logic.

Item	Change	Why
KioskProfileResult	Add id	● Blocking. Verified: the type carries full_name, certificates, skills, phone, cooperative_affiliation, member_since, programmes, attendance_count — no id . The NFC lookup literally cannot hand a trainee_id to the face check-in. buildKioskProfileResult already has profile.id in scope; it just isn't returned.
apiFetch	Attach status to thrown errors	● Verified: throws a bare Error with no status. 409 (already marked) is indistinguishable from a camera failure — students would see red and retry forever. Purely additive fix.
ESP32-CAM sketch	Add POST-upload route + mDNS	Currently a pull server (GET /capture); new flow needs it to push.
ESP32 sketch	LEDs, buzzer, debounced button, state machine, HTTP client	All new — no physical-feedback code exists anywhere in the repo.
Backend	devices table + requireDevice + 3 endpoints	Paths A/B only. Not needed for Path C.

⚠ **Firmware trap:** a non-match returns **HTTP 200** with {matched:false, fallbackToQr:true} — not an error. Branch on the field, not the status code, or every failed match reads as success.

Session dependency — not solved today

Verified from timetable_sessions: sessions are created only by an admin/trainer through the API; there is **no “active session” flag, no device binding, and no devices table anywhere**. The backend also **cannot currently reject** a check-in for being outside session hours — the route never reads starts_at / ends_at. Today a human types the session UUID into a text box; that is your remaining dependency.

Smallest fix: a devices table with institution_id, plus GET /device/current-session returning the session whose time window contains now(). The teacher still schedules the timetable in advance (already part of normal setup) but touches nothing on the day. **Tradeoff:** if no session is scheduled, the unit must refuse taps and show red — so the timetable must be populated before the demo.

Face positioning — the honest answer

You cannot detect “too close / too far / off-centre” today. Verified: human.detect(canvas) runs only on the already-captured frame, and the code reads just face.embedding. face.box is returned by Human but **currently discarded**. The backend receives no image at all, so it has zero positioning information.

Capability	Today	Notes
Face bounding box	⚠ unused	Available in-browser, code discards it
Detection before capture	✗ No	Needs continuous frames — the exact thing your WiFi could not sustain
Face size / position	⚠ post-capture only	Derivable from box
Confidence before submission	✗ No	Match is server-side by design (correctly)

MVP: yellow = “identified, waiting for capture”, blinking during processing. Both are states the system genuinely knows. **Do not label it “reposition.”**

Upgrade path: (1) *Post-capture retry guidance, nearly free* — you already compute “no face” after capture; add box size/centering checks on that same frame and return a distinct RETRY verdict instead of a red FAIL. (2) *On-device pre-capture detection, real future work* — ESP-WHO can run face detection on the CAM at QVGA, giving box size/position locally with no network traffic and **zero extra GPIO**. Budget it separately.

State machine

State	Green	Yellow	Red	Buzzer	Exit
OFFLINE	–	–	slow blink	–	WiFi up → IDLE
IDLE	–	–	–	–	UID read
IDENTIFYING	–	fast blink	–	–	lookup returns
READY_FOR_FACE	–	solid	–	short beep	button / 20 s timeout
CAPTURING / VERIFYING	–	slow blink	–	–	verdict / 15 s timeout
SUCCESS	solid 3 s	–	–	1 rising beep	auto → IDLE
FAILURE	–	–	solid 3 s	2 low beeps	auto → IDLE
DUPLICATE	blink 3 s	–	–	2 short beeps	auto → IDLE
TIMEOUT	–	–	blink 2 s	1 low beep	auto → IDLE

Two states you did not list but need: **DUPLICATE** (the 409 case — without it, already-marked students retry endlessly believing they failed) and **OFFLINE** (so a dead WiFi link is visible rather than looking like a frozen box).

Failure cases → device behaviour

Case	Action
Unknown card	Red + long beep, reset. No retry.
Taps twice / same student again	DUPLICATE state. Not an error.
Button before identification	Ignore , short error beep. Guard on state.
Button pressed repeatedly	Ignore while CAPTURING/VERIFYING (busy flag).
CAM offline / backend down	2 retries, then red + reset.
Student walks away after tapping	20 s timeout → TIMEOUT → IDLE.
Two students at once	Single-transaction lock; second tap ignored until reset.
Reboot mid-transaction	Boot to IDLE. Backend is authoritative — an unconfirmed transaction simply never happened.
Power cut <i>after</i> verification	Attendance already committed server-side; re-tap → DUPLICATE (correct).

Risk table

Issue	Likelihood	Impact	Mitigation
No server-side extraction exists	Certain	Fatal	Path B test → fall back to Path C
Weak WiFi / truncated frames	High (recurring)	High	QVGA, retries, AP within 2–3 m of unit
CAM brownout	Medium (already hit)	High	Own branch + 1000 µF + 3 A PSU
CAM IP changes	High (hit twice)	Medium	mDNS
409 reads as failure	Certain if unfixed	Medium	<code>apiFetch</code> status + DUPLICATE state
No session scheduled	Medium	High	Pre-seed timetable; OFFLINE/red state
Device key extracted from flash	Low at demo	Medium	NVS not source; rotatable; match stays server-authoritative
ESP32 HTTPS RAM pressure	Medium	Medium	<code>setInsecure()</code> for demo; pin CA later

Implementation plan

MUST HAVE — in order (1 and 2 are cheap and gate everything)

1. **Test tfjs-node on Linux (30 min).** This single result decides the whole architecture. Do it before anything else.
2. Add `id` to `KioskProfileResult` + status on `apiFetch` errors. Tiny, blocking, useful under every path.
3. Decide path → build device auth + 3 endpoints (A/B) **or** provision the Pi (C).
4. ESP32 firmware: RC522 on **GPIO 21**, LEDs, transistor-driven buzzer, debounced button, state machine.
5. ESP32-CAM: mDNS + upload route.
6. Power: 3 A PSU, star wiring, bulk caps.

RECOMMENDED

Post-capture RETRY guidance (yellow) · local NFC event queue for backend hiccups · HTTPS certificate pinning.

FUTURE

ESP-WHO on-device positioning (a real “reposition” LED) · full offline queue · OLED.

Offline resilience — keep it small. The repo’s offline support (`apps/web/src/offline/` , Capacitor Preferences, last-write-wins) is **browser-side only and does not apply to firmware**. A bounded in-RAM queue of unsent UID taps with retry is proportionate; more is a rewrite you don’t need.

Security. Store the device key in **NVS, flashed separately, never committed**. Be honest that a device key can be extracted from flash by anyone holding the hardware — treat it as *identification*, not strong authentication. What actually prevents spoofing is keeping the match server-side, which it already is. Rate-limit `/device/nfc-tap` per device and cap upload size.

SIH demo readiness — test in this order

1. **The extraction decision** — until Path B passes or C is provisioned, nothing else matters.
2. **WiFi at the actual venue.** Your link has repeatedly failed at 4.5 s-timeout for a 12 KB frame. Venue WiFi is typically worse and congested. **Bring your own travel router**; keep the AP 2–3 m from the unit. This is your single most likely live failure.
3. **Full cold boot** — power the sealed box from cold, confirm it reaches IDLE unattended, 5+ times.
4. **The duplicate path** — tap the same card twice on stage. It will happen.
5. **A scheduled session existing** for the demo time window.
6. **20 consecutive taps** — watch for heat, brownout, and stale-frame behaviour.
7. **A rehearsed fallback:** keep QR check-in ready. Judges forgive a fallback; they do not forgive a frozen box.

Two further challenges to the design:

1. The physical capture button may be unnecessary. Tapping the card already signals presence, so a 3-second auto-capture with a yellow countdown removes a GPIO, a debounce, and three race conditions. I would still keep the button — it lets the student control timing (fewer retries) and demos well — but it is a choice, not a requirement.

2. “No computer in the loop” is the requirement to relax first if Path B fails. It is costing the most for the least user-visible benefit.

Review based on direct inspection of: KioskNfcReader.tsx · KioskFaceCheckIn.tsx · packages/api-client · packages/shared-types · apps/api/src/routes/attendance.ts · publicProfile.ts · middleware/auth.ts · supabase/migrations · RC522 & PN532 sketches · ESP32-CAM sketch · CLAUDE.md · DECISIONS.md #5/#16/#21/#30/#32 · IMPLEMENTATION.md